

Customer Support Agent

Complete Amazon Bedrock AgentCore Project Review Notes

First-agent learning guide: what was built, why each step was required, and how the complete system works

Prototype -> Memory -> Gateway -> Security -> Evaluation -> Governance -> Harness -> Optimization

How to use these notes

Review the Project Story first, then study one lab at a time. For interview preparation, focus on the What / Why / How summaries, end-to-end flows, common troubleshooting section, and final interview explanation.

Prepared as a personal revision handbook

Contents

1. Project Story and Final Architecture
2. Lab 1 - Basic Agent and Local Tools
3. Lab 2 - Session Persistence and AgentCore Memory
4. Lab 3 - Gateway and Remote Lambda Tools
5. Lab 4 - Security, Identity, Sessions, and Observability
6. Lab 5 - Continuous Quality Evaluation
7. Lab 6 - Customer-Facing Chat Interface
8. Lab 7 - Cedar Policies and Tool Guardrails
9. Lab 8 - Zero-Code Agents with Harness
10. Lab 9 - Recommendations and A/B Testing
11. End-to-End Request Flows
12. Key Concept Comparisons
13. Troubleshooting Lessons
14. Git and Security Checklist
15. Interview and Revision Section

1. Project Story and Final Architecture

1.1 Project Goal

The project builds a production-style AI customer support agent for an e-commerce company using Amazon Bedrock AgentCore and the Strands framework. The agent answers questions, remembers users, accesses enterprise tools, enforces business policies, supports a browser chat experience, monitors quality, and improves through recommendations and controlled A/B testing.

- Answer product and return-policy questions
- Check warranties through an existing Lambda service
- Process refunds through a governed Gateway tool
- Remember customer facts across sessions
- Authenticate users with Cognito JWT tokens
- Monitor logs, traces, latency, memory, and tool calls
- Evaluate answer quality automatically
- Reuse the same tools and policies from a zero-code Harness agent
- Generate prompt/tool recommendations and validate them with A/B testing

1.2 Final Architecture

Customer -> Chat UI -> Cognito JWT -> AgentCore Runtime -> Secure Gateway -> Policy Engine -> Lambda Tools

Supporting services operate around this main flow: AgentCore Memory stores durable user context, CloudWatch stores OpenTelemetry traces and logs, AgentCore Evaluations scores interactions, and AgentCore Optimization proposes and validates improvements.

Core mental model

Runtime is the agent brain and execution environment. Gateway is the secure bridge to enterprise tools. Memory is durable user knowledge. Cognito proves identity. Policies decide what actions are permitted. CloudWatch explains what happened. Evaluations tell whether the result was good.

1.3 Project Evolution

Stage	Capability added
Lab 1	Model + prompt + local tools
Lab 2	Persistent user memory
Lab 3	Remote Lambda tools through Gateway
Lab 4	JWT security, session isolation, traces, and logs
Lab 5	Continuous quality monitoring

Stage	Capability added
Lab 6	Browser chat experience
Lab 7	Deterministic governance and sensitive-input guardrails
Lab 8	Zero-code Harness agent and human approval
Lab 9	AI-generated recommendations and A/B testing

2. Lab 1 - Basic Agent and Local Tools

What / Why / How	
What we did	Created a Strands-based CustomerSupport agent with a Bedrock model, a system prompt, local Python tools, and an Exa AI MCP client.
Why required	A model should not guess company product data or return policies. Tools ground answers in controlled business information.
How it works	The model reads the request, compares it with tool descriptions, calls the appropriate tool, receives structured data, and produces a customer-friendly response.

2.1 Main Components

- Foundation model: understands language and decides what to do
- System prompt: defines role, behavior, tone, and tool-use expectations
- Local tools: Python functions available inside the agent code
- AgentCore Runtime: hosts and executes the deployed agent
- MCP client: connects to externally hosted MCP tools such as Exa web search

2.2 Local Product Tool

The product-information tool searches a controlled product dictionary by ID, name, category, or keyword. This ensures prices and descriptions come from defined application data rather than model memory.

User asks about PROD-002 -> Model selects get_product_info -> Tool searches product data -> Model formats response

2.3 Local Return-Policy Tool

The return-policy tool accepts a product category and returns the correct return window, conditions, and refund method. The model should first identify the category, then call this tool rather than inventing a policy.

2.4 Exa AI MCP

The Exa MCP client demonstrates that an agent can combine local functions with tools provided by a remote MCP server. MCP standardizes tool discovery and invocation so the agent focuses on tool purpose rather than backend implementation.

Lab 1 takeaway

A useful agent is not just an LLM. It is a model guided by a system prompt, connected to trustworthy tools, and hosted in an execution environment.

3. Lab 2 - Session Persistence and AgentCore Memory

What / Why / How	
What we did	Added AgentCore Memory with semantic fact extraction, conversation summarization, and a memory session manager keyed by user ID and session ID.
Why required	The basic agent forgets customers when a conversation ends. Customer support requires continuity, personalization, and separation between users.
How it works	Current-session history stays with the active session, while AgentCore Memory stores durable facts and summaries outside the microVM for later retrieval.

3.1 User, Session, and MicroVM

- User ID identifies the person or actor.
- Session ID identifies one conversation.
- One user can have many sessions.
- Each active session runs in an isolated lightweight microVM.
- The microVM holds temporary in-process conversation state.
- AgentCore Memory is separate and can survive session termination.

Session Persistence	AgentCore Memory
Within one session	Across multiple sessions
Live conversation state	Durable extracted facts and summaries
Temporary; ends with session/microVM	Persists according to memory retention policy
Built into Runtime session handling	Requires AgentCore Memory configuration

3.2 Memory Namespaces

Semantic user facts are stored under a user-scoped namespace such as `/users/{actorId}/facts`.
 Conversation summaries are stored under a user-and-session-scoped namespace such as `/summaries/{actorId}/{sessionId}`.

3.3 Example

Sarah says: I prefer email -> Memory extracts preference -> New session starts -> Agent retrieves Sarah facts -> Agent personalizes response

Why both are needed

Session persistence makes a live conversation natural. AgentCore Memory makes future conversations personalized. Replacing either one with the other would be inefficient or incomplete.

4. Lab 3 - Gateway and Remote Lambda Tools

What / Why / How	
What we did	Exposed an existing warranty-check Lambda through AgentCore Gateway as the MCP tool check_warranty.
Why required	Enterprise logic already exists in Lambdas and APIs. Copying that logic into every agent creates duplication, inconsistent behavior, and maintenance risk.
How it works	A tool schema describes the capability. Gateway maps the MCP tool call to Lambda parameters, invokes the function, and returns the result to the agent.

4.1 Why the Tool Schema Is Necessary

- Tool name tells the model what capability exists.
- Natural-language description tells the model when to use it.
- Input schema tells the model which arguments are required and their types.
- Required fields prevent incomplete tool calls.
- The schema describes functionality; it does not create the underlying business logic.

4.2 ARN and Parameter Store

The Lambda ARN is the unique address of the warranty function. It is retrieved from SSM Parameter Store instead of being hardcoded, making the configuration easier to manage across environments.

4.3 Tool Types Used Together

- Local: get_product_info and get_return_policy
- Direct remote MCP: Exa web search
- Gateway MCP: check_warranty backed by Lambda

4.4 End-to-End Warranty Flow

User asks warranty -> Agent selects check_warranty -> MCP client calls Gateway -> Gateway invokes Lambda -> Lambda returns status -> Agent answers

Gateway takeaway

Gateway turns existing enterprise services into discoverable agent tools while keeping the original service implementation and ownership unchanged.

5. Lab 4 - Security, Identity, Sessions, and Observability

5.1 Session Continuity and Isolation Test

The lab separated three concepts: same-session persistence, cross-session isolation, and durable AgentCore Memory. Using Alex in a fresh session demonstrated temporary context; using Sarah in a new session demonstrated durable memory recall.

1. Same session: Alex says what was purchased and the agent remembers the follow-up.
2. New session: the live history is not visible, proving session isolation.
3. Existing user memory: Sarah is recognized in a new session because durable facts were aggregated earlier.

5.2 Observability

What / Why / How	
What we did	Used OpenTelemetry traces, CloudWatch logs, CLI trace commands, live log streaming, and keyword/error searches.
Why required	When an agent gives a wrong answer, developers need to know whether the failure came from memory, model reasoning, tool selection, Gateway, Lambda, or the final response.
How it works	Every request produces spans that record the request journey. Logs contain detailed content, warnings, errors, tool arguments, and tool responses.

Traces	Logs
Structure and timing of a request	Detailed messages and content
Shows spans and call order	Shows warnings, errors, and printed output
Useful for latency and execution path	Useful for exact inputs, outputs, and failure details

5.3 From IAM to Cognito JWT

IAM protects service-to-service calls but is not suitable for giving every customer AWS credentials. Cognito allows real users to authenticate with a username and password and receive a JWT access token.

User signs in -> Cognito issues JWT -> Client sends Bearer token -> Runtime validates token -> Agent extracts username -> Memory uses username as actor ID

5.4 Cognito Configuration Values

- Discovery URL: tells AgentCore where to obtain issuer metadata and signing keys.
- Client IDs: identify which applications are allowed to obtain accepted tokens.

- Pool ID: identifies the Cognito User Pool for user-management operations.
- Authorization header allowlist: allows the token to reach application code when required.

5.5 Securing the Gateway

Runtime and Gateway are separate endpoints. Both were secured with the same Cognito identity provider. The Runtime forwards the incoming Authorization header to the Gateway MCP client, creating end-to-end identity propagation.

Client JWT -> Runtime JWT validation -> Authorization header forwarded -> Gateway JWT validation -> Tool invocation

Security distinction

Authentication answers “Who are you?” Authorization answers “What are you allowed to do?” Lab 4 established identity; Lab 7 later added action authorization.

6. Lab 5 - Continuous Quality Evaluation

What / Why / How	
What we did	Created an online evaluation monitor using GoalSuccessRate, Correctness, and ToolSelectionAccuracy at a 100% workshop sampling rate.
Why required	A technically working agent may still fail customer goals, hallucinate facts, or select the wrong tool. Quality must be measured, not assumed.
How it works	AgentCore samples live sessions, sends the traces to built-in LLM-as-a-Judge evaluators, stores scores, and exposes aggregates and detailed explanations.

6.1 Evaluators

- Goal Success Rate: Did the interaction solve the customer's actual problem?
- Correctness: Was the information factually accurate and supported?
- Tool Selection Accuracy: Did the model choose the tool best suited to the request?

6.2 Online vs On-Demand Evaluation

Online Evaluation	On-Demand Evaluation
Continuously scores sampled live interactions	Runs against a chosen historical lookback window
Useful for ongoing monitoring	Useful for immediate analysis and comparison
Configured once and left active	Started manually when needed

6.3 Interpreting Scores

- 80-100%: excellent; monitor and maintain.
- 60-80%: acceptable but needs review of low-scoring sessions.
- Below 60%: investigate root causes and improve prompt, tools, or data.

6.4 Why Detailed Evaluation Logs Matter

Aggregate scores describe overall performance. Detailed evaluation logs explain why a particular interaction received its score, such as an invented product ID, an unnecessary clarification question, or a correct warranty-tool call.

Evaluation takeaway

AgentCore Evaluations gives the agent an automatic report card and creates the evidence used later by AgentCore Optimization.

7. Lab 6 - Customer-Facing Chat Interface

What / Why / How	
What we did	Ran a browser-based frontend that authenticates the test user, sends prompts with a bearer token, maintains a session, and displays responses.
Why required	CLI invocation is suitable for developers but not customers. A frontend is required to turn the backend agent into a usable application.
How it works	The frontend signs in through Cognito, sends the JWT and session information to AgentCore, streams the response, and displays the conversation.

Browser -> Cognito login -> JWT token -> Runtime invocation -> Streaming response -> Chat display

- A hard browser refresh may be required after redeploying because cached frontend assets or authentication state can become stale.
- The frontend and the CLI commonly run in separate terminal tabs.
- The project root remains ~/workshop/CustomerSupport; the frontend runs from its own subfolder.

8. Lab 7 - Cedar Policies and Tool Guardrails

What / Why / How	
What we did	Added a refund Lambda tool, created a Policy Engine, attached it to the secure Gateway in ENFORCE mode, and wrote Cedar policies for refund limits and warranty access.
Why required	An authenticated customer should not automatically be allowed to perform every business action. High-impact tool calls require deterministic business authorization outside model reasoning.
How it works	Every Gateway tool call is evaluated against Cedar policies before execution. Default deny blocks actions unless a permit rule matches; forbid rules can override permits.

8.1 Policy Engine and Cedar

- Policy Engine: container and evaluator for policies.
- Cedar policy: declarative permit or forbid rule.
- ENFORCE: denied requests are blocked.
- LOG_ONLY: policy results are recorded but do not block traffic.
- Default deny: no explicit permit means the action is rejected.

8.2 Refund Limit

Agent requests refund -> Gateway receives amount -> Policy checks amount < 100 -> Allow or deny -> Lambda runs only if allowed

The policy permits automated refunds below \$100 and blocks refunds of \$100 or more. This keeps a probabilistic model from directly performing high-value financial actions.

8.3 Warranty Permit

Because the Policy Engine uses default deny, the existing warranty tool also required an explicit permit policy. This preserved safe read-only functionality after governance was enabled.

8.4 Sensitive Refund-Reason Guardrail

A semantic forbid policy checks the exact tool argument context.input.reason for email addresses. The original user message is not the enforcement target; the guardrail evaluates what the model actually sends to Gateway.

User message -> Model constructs reason -> Gateway guardrail checks EMAIL -> Forbid overrides permit -> Lambda blocked if sensitive

- A \$60 refund may pass the amount rule but still be blocked if the reason contains an email.
- If the model removes the literal address and sends only a generalized reason, the guardrail may not match.

- Logs must be checked to verify the exact tool arguments and whether the guardrail triggered.

Governance takeaway

The model proposes actions. Gateway policies make the final deterministic authorization decision. Guardrails additionally control what data is allowed to enter a tool.

9. Lab 8 - Zero-Code Agents with Harness

What / Why / How	
What we did	Created OrderResearchAgent declaratively using a model, system prompt, Code Interpreter, existing Gateway tools, OAuth outbound authentication, filesystem access, model overrides, and an inline human-approval function.
Why required	Some agents need standard model + prompt + tools orchestration rather than custom Python runtime code. Harness reduces implementation and deployment effort.
How it works	AgentCore manages the orchestration loop and a session microVM. The Harness configuration declares the model, prompt, tools, memory, environment, and optional skills.

9.1 Runtime Agent vs Harness Agent

Coded Runtime Agent	Harness Agent
Custom main.py and dependencies	Declarative harness.json
Maximum orchestration control	Managed standard orchestration
Best for custom request logic	Best for rapid model + prompt + tools agents
Application code owns behavior	Harness service owns execution loop

9.2 OAuth Outbound Authentication

The coded Runtime forwarded a user JWT to Gateway. The zero-code Harness had no inbound user token, so it obtained its own Cognito machine token using the OAuth client-credentials flow.

Harness -> **OAuth credential provider** -> **Machine token** -> **Secure Gateway** -> **Governed tools**

9.3 Code Interpreter and Shell Access

- Create and edit reports and datasets.
- Run deterministic Linux commands in the session environment.
- Inspect generated files.
- Use shell results instead of relying on model estimation.

9.4 Filesystem Persistence

Files persist within the same Harness session, enabling iterative report creation. Optional session-storage mounts can preserve artifacts across stop/resume cycles for the same session ID.

9.5 Model Override

A different model can be selected for one invocation without redeploying, which supports cost, latency, and quality comparisons while preserving the Harness configuration and session state.

9.6 Human-in-the-Loop

\$200 refund request -> Automated policy denial -> Agent calls approve_exception -> Harness pauses -> Human approves/denies -> Agent resumes

Inline functions are declarative pause points. The external client performs the real approval logic and sends a tool result back. This pattern supports Slack approvals, queues, dashboards, supervisor agents, or compliance review.

9.7 Advanced Harness Features

- Persistent storage mount: keeps reports available beyond a single active execution cycle.
- Custom container: supplies pre-installed software and libraries while Harness still manages the agent loop.
- Agent skills: reusable instructions, scripts, and references that add specialized capabilities without fine-tuning.

10. Lab 9 - Recommendations and A/B Testing

What / Why / How	
What we did	Used real traces and GoalSuccessRate evaluations to generate system-prompt and tool-description recommendations, packaged prompts as config bundles, created a dedicated IAM A/B Runtime, split Gateway traffic, and measured statistical results.
Why required	Evaluation reveals the current score but does not automatically explain how to improve. Recommendations propose changes, and A/B testing verifies whether those changes help real users.
How it works	Optimization analyzes success/failure patterns. Gateway assigns sessions to control or treatment bundles. Online evaluation scores each variant, and statistics determine whether the observed difference is likely real.

10.1 Recommendation Loop

Traces -> Evaluation scores -> Failure patterns -> Recommended prompt/tools -> A/B test -> Promote winner -> New baseline

10.2 System-Prompt Recommendation

The optimizer preserves the original intent and adds concrete guidance based on recurring failures, such as using user_context, searching products proactively, minimizing unnecessary clarification, and requesting approval before real-world actions.

10.3 Tool-Description Recommendation

The optimizer studies tool-selection traces and suggests clearer descriptions. Better descriptions tell the model when to use a tool, which inputs are preferred, which related tool should run first, and whether tools can run in parallel.

10.4 Why a Dedicated CustomerSupportAB Runtime

The production Runtime accepts Custom JWT. The Gateway http-runtime target invokes a Runtime using IAM/SigV4. A dedicated memoryless CustomerSupportAB Runtime retained IAM authentication so the Gateway could invoke it without weakening production security.

Client JWT -> Gateway -> Control or Treatment bundle -> SigV4 -> CustomerSupportAB

10.5 Config Bundles

A config bundle is a versioned, immutable snapshot of agent configuration. Control contains the baseline prompt; treatment contains the recommended prompt. The code, model, tools, and Runtime remain the same so the prompt is the only major experimental variable.

10.6 Traffic Split

An 80/20 test routes 80% of new sessions to control and 20% to treatment. Assignment happens per session so all requests in a session remain consistent with one variant.

10.7 Why LOG_ONLY During the Test

The Lab 7 Policy Engine had no explicit permit for the new A/B HTTP target and would block traffic in ENFORCE mode. LOG_ONLY records policy decisions without blocking the experiment. Production governance should be restored or replaced with an explicit permit after testing.

10.8 Statistical Decision

- Treatment mean higher than control mean: potential improvement.
- Positive percent change: treatment moved in the desired direction.
- p-value below 0.05: evidence that the difference is unlikely to be random.
- Confidence interval fully above zero: improvement direction is consistently positive.
- Check every evaluator; improving goal success while reducing correctness may not be acceptable.

Optimization takeaway

A recommendation is a hypothesis. A/B testing converts that hypothesis into measurable evidence before production rollout.

11. End-to-End Request Flows

11.1 Product Information

User asks product question -> Runtime -> get_product_info -> Product data -> Final answer

11.2 Warranty Check

JWT client -> Runtime -> Gateway MCP client -> Gateway JWT + policy check -> Warranty Lambda -> Response

11.3 Small Refund

Refund request -> Runtime selects tool -> Gateway -> Amount policy permits -> Refund Lambda -> Confirmation

11.4 Large Refund

Refund request >= \$100 -> Gateway -> Cedar default/limit decision -> Denied -> Agent explains or escalates

11.5 Sensitive Refund Reason

Refund reason contains email -> Gateway guardrail -> EMAIL confidence threshold met -> Forbid -> Lambda not invoked

11.6 Human Approval

Automated denial -> Inline approval tool -> Harness pause -> Human decision -> toolResult -> Agent resumes

11.7 Quality Improvement

Live sessions -> CloudWatch traces -> Evaluations -> Recommendations -> Control/Treatment -> Statistics -> Winner

12. Key Concept Comparisons

12.1 Runtime vs Gateway

AgentCore Runtime	AgentCore Gateway
Hosts the agent and model interaction	Hosts and routes remote tool access
Understands user intent and chooses tools	Authenticates, authorizes, and invokes backends
Uses memory and creates final response	Returns tool results to agent
Agent brain/execution environment	Secure bridge and traffic controller

12.2 Authentication vs Authorization vs Guardrails

Layer	Question	Project mechanism
Authentication	Who is calling?	Cognito JWT
Authorization	What may the caller/agent do?	Cedar policies
Guardrail	What content may enter the tool?	Sensitive-information semantic check

12.3 Local Tool vs Gateway Tool vs MCP Tool

Local Tool	Gateway Tool
Python function in agent project	Remote enterprise service exposed through Gateway
Fast and easy for small logic	Central ownership and reuse across agents
Deployed with agent	Service can evolve independently
Example: return policy	Example: warranty Lambda

A direct MCP tool is hosted by an external MCP server and connected without using your Gateway; Exa web search is the project example.

12.4 Memory vs Knowledge Base

Memory stores user- and session-related context such as preferences and summaries. A knowledge base would store reusable organizational information for retrieval across many users. This project focused on Memory rather than an enterprise knowledge base.

13. Troubleshooting Lessons

13.1 Linux vs Windows Commands

- The Workshop Code Editor terminal runs Linux Bash even when the physical laptop runs Windows.
- Linux uses backslash for line continuation; PowerShell uses a backtick.
- Linux variable assignment uses NAME=value without spaces.
- If the prompt looks like participant:~/workshop/CustomerSupport\$, use the macOS/Linux tab.

13.2 Project Root and Folder Structure

The outer CustomerSupport folder is the project root. app/CustomerSupport contains the deployed Runtime application code. Additional agents and Harnesses may appear in separate app subfolders. Do not merge or move these folders because agentcore.json points to their code locations.

13.3 HTML Tags Appearing in Chat

URLs copied from the workshop browser sometimes appeared as HTML anchor tags when pasted into chat, even when the real VS Code file contained plain URLs. Validate the real file with JSON/Python parsers rather than relying only on pasted formatting.

13.4 Similar-Looking Characters

Cognito Pool IDs and Client IDs may contain both the number 1 and lowercase letter l. Copy values directly from SSM and compare exact strings. A single incorrect character can cause discovery or client-validation failures.

13.5 Duplicate Gateway Targets

Removing and recreating Gateway resources can leave unassigned or duplicated targets in local configuration. Inspect agentcore.json, make sure targets appear only in the intended Gateway, validate, and deploy in the workshop's required order.

13.6 Code Signature Mismatch

When main.py changed to call get_gateway_mcp_client(auth_header), client.py also had to accept auth_header. Function definition and call signatures must evolve together.

13.7 Deployment vs Runtime Errors

- CDK synth/deployment failure occurs before application code runs and usually points to infrastructure configuration.
- Python import or TypeError occurs when the deployed application starts and usually points to code/dependency mismatch.
- Authorization method mismatch means the caller used JWT while the endpoint expected IAM, or vice versa.
- UsernameExistsException means the Cognito user is already present; reset the password or reuse the user rather than creating a duplicate.
- No supported spans for evaluation means traces are missing, too new, or not in the expected scope/log group.

13.8 Preferred Validation Sequence

4. Save all files.
5. Validate JSON syntax.
6. Compile Python files to catch indentation/import syntax issues.
7. Run agentcore validate.
8. Deploy with verbose output.
9. Run agentcore status.
10. Test one simple local-tool request.
11. Test one Gateway-tool request.
12. Inspect traces and logs if behavior differs from expectations.

14. Git and Security Checklist

14.1 Before Committing

- Run git status and review every staged file.
- Do not commit AWS access keys, secret keys, session tokens, JWT access tokens, client secrets, .env files, or credential caches.
- Ignore virtual environments, __pycache__, generated trace files, temporary load-test output, and local deployment artifacts when appropriate.
- Respect company policy before uploading code from a corporate device or workshop environment. Prefer a private repository when required.

14.2 Recommended Review Commands

```
git status  
  
git diff --cached  
  
git remote -v  
  
git log --oneline -5
```

14.3 Authentication to GitHub

GitHub HTTPS pushes require a supported credential method such as a personal access token or an authenticated GitHub CLI session. Account passwords are not accepted for Git operations. If the VS Code credential-helper socket fails, reauthenticate instead of embedding tokens in the remote URL.

Security reminder

Never paste GitHub tokens, Cognito client secrets, AWS credentials, or JWT values into source files, screenshots, chat messages, or repository history. Rotate any secret that may have been exposed.

15. Interview and Revision Section

15.1 60-Second Project Explanation

I built an enterprise customer support agent using Amazon Bedrock AgentCore and Strands. The agent uses local tools for product and return-policy data, AgentCore Memory for cross-session user personalization, and AgentCore Gateway to expose existing warranty and refund Lambdas as MCP tools. I secured both Runtime and Gateway with Cognito JWT authentication, forwarded the user identity into Memory, and added CloudWatch traces and logs for observability. I implemented Cedar policies to restrict automated refunds, explicitly allow warranty checks, and block sensitive email data in refund tool arguments. I configured continuous evaluations for goal success, correctness, and tool selection, created a zero-code Harness agent that reused the same governed Gateway, added human approval for high-value refunds, and used optimization recommendations plus A/B testing to validate a better system prompt before rollout.

15.2 Short Answers to Common Questions

Question	Answer
Why use Gateway?	To expose existing enterprise services as standardized, discoverable agent tools without duplicating backend logic.
Why both session persistence and Memory?	Session persistence keeps the current conversation coherent; Memory preserves useful user facts across different sessions.
Why secure both Runtime and Gateway?	They are separate endpoints. Securing only one leaves another path that could be called independently.
Why policies outside the agent code?	Deterministic rules cannot be bypassed by model mistakes or prompt injection and can be managed centrally.
Why Harness?	It enables rapid zero-code agents when model + prompt + tools are sufficient and custom orchestration is unnecessary.
Why A/B testing?	A recommended prompt is only a hypothesis; live controlled measurement is required before production rollout.

15.3 Final Revision Checklist

- Can I explain Runtime, Gateway, Memory, MCP, Cognito, Policy Engine, Evaluations, Harness, and Config Bundles in one sentence each?
- Can I draw the warranty request flow from user to Lambda and back?
- Can I explain session persistence vs AgentCore Memory?
- Can I explain authentication vs authorization vs guardrails?
- Can I explain why the A/B Runtime uses IAM while the production Runtime uses JWT?
- Can I describe one error I faced and how I diagnosed it?
- Can I explain what is probabilistic (model/semantic guardrail) and what is deterministic (Cedar authorization, shell commands)?

Final project message

This project is a complete production lifecycle, not only a chatbot: build, ground, remember, connect, secure, govern, observe, evaluate, simplify, optimize, and validate.